

A Discord-Based System for Human-in-the-Loop PDDL Modeling, Solving, and Validation through Natural Language

Adam Neto*, Jawad Ahmed*, Marcus Tantakoun, Christian Muise

School of Computing, Queen’s University
{21amn13, 19jaa14, 20mt1, christian.muise}@queensu.ca

Abstract

Traditional planning workflows often require prerequisite knowledge of PDDL modeling, along with the implementation of validation and planning tools. This steep learning curve creates a significant barrier to entry for new users. To address this, we introduce the **Interactive Planning Agent (IPA)**: an LLM-based planning system designed to make the generation of PDDL models accessible through natural language conversations on Discord. Unlike traditional chatbots, multiple users can collaborate with the system within a shared session, allowing teams to iteratively contribute to and refine PDDL domains and problems, and use external planners in real time. This architecture connects Model-Context Protocol (MCP) back-end services with a front-end Discord bot. This system relies on interactive model management and external services such as L2P (*l2p_mcp*) and Planning-as-a-Service (PaaS) that support human-in-the-loop planning.

Introduction

PDDL (McDermott et al. 1998) serves as a foundational language for symbolically modeling and solving complex problems across a wide variety of domains. However, the traditional workflow can be inaccessible for many users due to the expertise needed for manual PDDL authoring, validation, and execution with planners (Gestrin, Kuhlmann, and Seipp 2024). This creates a barrier to entry for those who lack the technical background to use it effectively.

The application of Large Language Models (LLMs) with automated planning model acquisition has seen considerable growth in recent years (Katz et al. 2026), with increasing efforts to deploy such systems in practical settings. However, current systems, such as *PDDL-GenAI* (Wang et al. 2025) and *Planning Copilot* (Benyamin et al. 2025), remain narrowly focused on direct model generation or planner execution in isolation. This offers limited support and flexibility to users who lack the prerequisite domain knowledge needed to bootstrap a planning model from scratch, or require guided explanations of the model being generated in order to meaningfully review and refine it. While general-purpose conversational systems, such as ChatGPT, can assist users in drafting PDDL models, they operate in isolation from the broader planning toolchain (e.g., lacking the ability to invoke syntax checkers, classical planners, or validate plans).

*These authors contributed equally.

To address these issues, this work introduces the **Interactive Planning Agent (IPA)**: a human-in-the-loop (HITL) system that empowers users to engage with planning models and tools through natural language on Discord via a “chat-bot” style interface. Rather than requiring users to commit to single-shot generation of NL-PDDL models, IPA allows them to converse iteratively with the agent, supporting incremental updates, validation, and solver interaction in an accessible end-to-end workflow. This demonstration highlights our system’s extensibility across diverse planning tasks, advancing accessible automated planning as a meaningful contribution to the ICAPS community.

System Overview

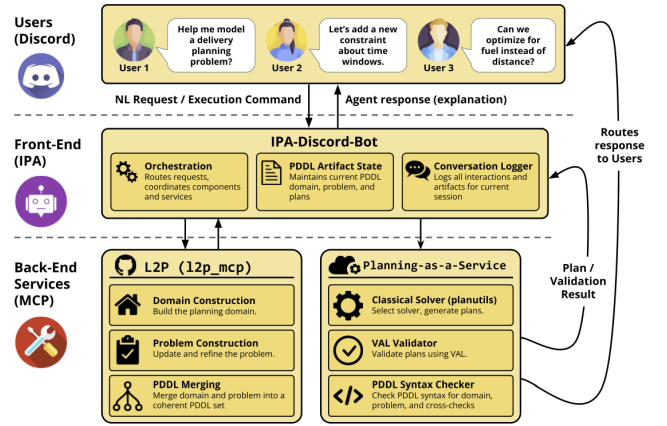


Figure 1: Overview of IPA Workflow

The **Interactive Planning Agent** system consists of three main components: 1) *IPA-Discord-Bot* serves as the Discord-based front-end, which allows users to interact with the system via NL. It manages orchestration of user requests, maintains current PDDL artifacts, and handles shared conversation logging; 2) *l2p_mcp*, built upon *L2P* (Tantakoun, Muise, and Zhu 2025), is the MCP-based back-end responsible for incremental construction and revision of PDDL domains and problems based on LLM-supported NL requests; 3) *planning-as-a-service* (PaaS) hosts solver-oriented back-end services through MCP, exposing available tools from the *planutils* (PU) toolkit.

As seen in Figure 1, the end-to-end flow starts with one or more users issuing a request via the Discord bot (LLM-wrapped agent), that decides the appropriate back-end service for processing. The resulting artifacts are then returned through the bot to the user, enabling an interactive workflow that eliminates the need for manual labor. To support continuity across interactions, the system maintains persistent memory of prior conversations, enabling shared context-aware responses and reproducible planning workflows.

Table 1: Key IPA Discord Bot Commands

Discord Command	Description
!domain <request>	Generate a PDDL domain from a NL request.
!problem <request>	Generate a PDDL problem from a NL request.
!plan <request>	Uses selected solver to generate plan.
!edit <artifact>	Revises artifact + preserves workflow state.
!show <artifact>	Display the current working artifact in chat.
!explain <artifact>	Explain the current artifact in NL.
!undo <artifact>	Restore the previous version of an artifact.
!validate_plan	Validate domain/problem/plan triple via PaaS.
!autovalidate	Loop repairs via VAL until the plan passes.
!collab	Toggle shared collaboration mode in channel.
!thread [topic]	Create private thread for bot interaction.
/usesolver <solver>	Set solver via PaaS (manual or LLM decides).
/setkey <key> <prov.>	Set API key and LLM provider.
/use <model.id>	Set the LLM model for your requests.

Table 1 summarizes the main commands available through the IPA Discord interface. Users may freely converse with the agent in natural language to provide context, clarify requirements, or explore the planning problem before invoking any commands. Once sufficient context has been established, executing a command triggers the agent to route the request to the appropriate MCP back-end service.

Demonstration Plan

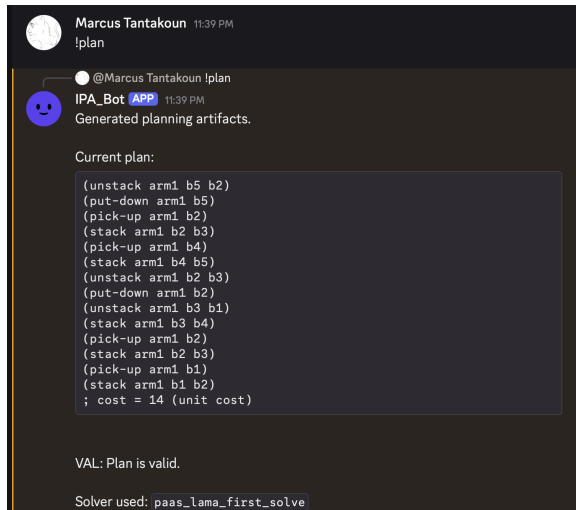


Figure 2: Interface Demo Figure

The live demo will present a user interacting with the bot through a short end-to-end planning workflow in Discord.

Figure 2 illustrates a !plan invocation: the agent selects an appropriate solver from the available PaaS tools, generates a plan from the current domain and problem, and automatically validates the output. The demo will then show inspection of the generated artifacts, an incremental NL revision to the problem specification, and regeneration of the plan after the update has been merged by l2p_mcp. To highlight correctness checks within the same workflow, the user will also invoke syntax and plan validation functionality through PaaS. Mirroring the existing Demo Video,¹ the demonstration is designed to make the layered interaction between conversational input, structured model revision, validation, and solver access visible to the audience.

Discussion for the Demo Track

- **Robustness:** IPA combines LLM flexibility with deterministic back-end tools for robust, consistent request handling. PDDL artifact generation is managed by l2p_mcp, which parses, merges, and normalizes schemas via the L2P toolkit. Syntax and plan correctness are validated through the VAL validator and PDDL syntax tool, both exposed via PaaS. This provides a reliable safeguard against malformed artifacts.
- **Generalization:** IPA operates across a wide variety of planning domains through the capabilities of the L2P library. Once an initial plan is formed, the domain and problem are stored in PDDL format and can be viewed, edited, or validated through the same Discord workflow at any time. The MCP-based architecture cleanly separates front-end user interaction from back-end tool execution, making the system domain-agnostic by design.
- **Scalability:** The system scales with problem complexity through incremental human-in-the-loop editing. Rather than regenerating full plans from scratch, users make targeted revisions and validate after each step. This workflow proves substantially more reliable than unconstrained generation for larger problem instances.
- **Deployment Effort:** Both IPA-Discord-Bot and l2p_mcp can be deployed locally as persistent Docker services. The PaaS MCP endpoint is publicly hosted,² meaning solver and validation capabilities require no local setup. In addition, other MCP clients may also connect directly to PaaS, extending the reach of the system beyond IPA itself.

Conclusion

Our work presents the Interactive Planning Agent: a practical way to make automated planning more accessible through the use of a Discord bot. Using our system, plans can be easily generated and modified through conversational workflows which support revision, validation, and solving—all through a single interface. More broadly, this demo showcases how MCP-connected services and structured PDDL workflows lower the barrier to automated planning.

¹Demo Video: <https://tinyurl.com/IPA-Discord-Bot-Video>

²PaaS MCP Endpoint: <https://solver.planning.domains/mcp>

References

- Benyamin, Y.; Mordoch, A.; Shperberg, S. S.; and Stern, R. 2025. Toward PDDL Planning Copilot. *arXiv preprint arXiv:2509.12987*.
- Gestrin, E.; Kuhlmann, M.; and Seipp, J. 2024. NL2Plan: Robust LLM-Driven Planning from Minimal Text Descriptions. *CoRR*, abs/2405.04215.
- Katz, M.; Kokel, H.; Srinivas, K.; and Sohrabi, S. 2026. Planning in the LLM Era: Building for Reliability and Efficiency. *CoRR*, abs/2605.21902.
- McDermott, D.; Ghallab, M.; Howe, A.; Knoblock, C.; Ram, A.; Veloso, M.; Weld, D.; and Wilkins, D. 1998. PDDL – The Planning Domain Definition Language. Technical report, AIPS-98 Planning Competition Committee.
- Tantakoun, M.; Muise, C.; and Zhu, X. 2025. L2P: A Python Toolkit for Automated PDDL Model Generation with Large Language Models.
- Wang, J.; Peng, R.; Chen, T.; Zeng, C.; Wang, W.; Huang, S.; and Lipovetzky, N. 2025. PDDL-GenAI – A Plugin for LLM-Based Support in Planning Domain Modeling. In *ICAPS-25 System Demonstrations*.